

Sourcerer — Execution Plan

What this project is, in one sentence: Sourcerer is a system where you can upload a document, ask it questions, and get answers that are actually backed by the text — built step by step, so at every stage you have something real that works, and you can prove (with numbers) that each new piece you add actually makes it better.

How to use this document: Each stage below has two parts —

- **What it does (plain language):** what a regular person would see or experience at this point.
- **What you're building (technical):** what you actually do to get there.

Don't move to the next stage until the current one works. That's the whole rule.

Stage 0 — The Basic Pipeline

What it does (plain language): You give the system a document and a question. It reads through the document, finds the part that seems relevant, and writes you an answer based on that part. Nothing fancy yet — think of it as a very basic "ask questions about this file" tool that only you can use, from your own computer, typed in through a plain black screen (no fancy webpage yet).

What you're building (technical):

- Parse a PDF into plain text
 - Split the text into small chunks (a few hundred words each)
 - Turn each chunk into a vector (an embedding) using a pre-trained model — no training required
 - Store those vectors so they don't have to be recreated every time you run the program
 - When a question comes in: turn the question into a vector too, find the closest matching chunks, hand them to an LLM, return the answer
 - Wrap file-reading and API calls in basic error handling so a bad file or failed request doesn't crash the whole thing
 - Start a running notes file (`decisions.md`) — log every real choice you make from here on (chunk size, model used, file size limits, why you picked them). You'll need this later.
-

Stage 0.5 — A Simple Screen to Use It

What it does (plain language): Instead of typing into a black command screen, you now get an actual webpage: a box to type your question, a little "searching..." message while it thinks, and then the answer — with the original piece of text it came from shown underneath, so you can double check it isn't making things up.

What you're building (technical):

- Build a Streamlit app (a quick way to turn a Python script into a webpage without needing to be a web developer)
 - Wire it to your Stage 0 logic — no new backend work, just a front end sitting on top
-

Stage 1 — Building a Report Card

What it does (plain language): This stage doesn't change what the user sees at all. Behind the scenes, you're building yourself a "test" — a list of 20-30 questions where you already know the correct answer should come from a specific part of the document. This is how you'll know, going forward, whether each new feature you add is actually helping or just adding complexity.

What you're building (technical):

- Create 20-30 (question, correct source chunk) pairs — can be bootstrapped by having an LLM write a question from a known chunk, then sanity-check a sample by hand
 - Write simple scoring functions: how often is the correct chunk found in the top results? (Recall), how high up is it? (MRR)
 - Run your Stage 0 pipeline against this test sheet and record the starting scores in `decisions.md`
-

Stage 2 — Adding Keyword Search

What it does (plain language): Right now, the system finds text based on "meaning" — but sometimes it misses obvious things, like an exact name, code, or rare term, because it's only thinking about general meaning rather than exact words. This stage adds a second, simpler kind of search that looks for literal word matches, and combines it with the first one, so the system catches both kinds of matches.

What you're building (technical):

- Add a keyword-based search index (BM25) alongside your existing vector search

- Combine the two ranked result lists using Reciprocal Rank Fusion (RRF) — a simple, standard way to merge two rankings without messing with score scales
 - Re-run your Stage 1 test sheet, record whether Recall/MRR improved
-

Stage 3 — Adding a Careful Second Look (Reranking)

What it does (plain language): After the system finds a batch of "maybe relevant" pieces of text, this stage adds one more, more careful pass: it takes the top candidates and re-orders them by how genuinely relevant they are to your specific question, keeping only the best handful. Think of it as a first-round of quick guesses, followed by someone who actually reads them closely and picks the best ones.

What you're building (technical):

- Take the top ~30-50 fused results and score them with a cross-encoder reranking model (e.g. `bge-reranker-v2-m3`)
 - Keep only the top 5-8 after reranking — this becomes what actually gets passed to the LLM
 - Re-test against your sheet, record the new scores
-

Stage 4 — Improving the Question Itself

What it does (plain language): Sometimes the way a person phrases a question doesn't match the wording used in the document. This stage adds a step where, before searching, the system quietly rewrites or expands your question into wording that's more likely to match what's actually in the text — then searches using that improved version instead.

What you're building (technical):

- Add a query rewriting step (start simple — HyDE: ask the LLM to write a hypothetical answer, then search using that instead of the raw question)
 - Log both the original and rewritten query for debugging
 - Re-test, record scores
-

Stage 5 — The Trust Check

What it does (plain language): This is the safety net. After the system writes an answer, it now double-checks itself: "does this answer actually match what the source text says?" If it's confident and well-supported, you see the answer normally. If it isn't well-supported, instead of guessing, the system honestly tells you "not enough information found" — rather than sounding confident about something it isn't sure of. On the screen, you'll now see a small tag next to each answer, like "high confidence" or "low confidence."

What you're building (technical):

- Add an LLM-as-judge step: given the answer and the source chunks used, ask "is this fully supported?" and get back a score
 - If below a threshold, replace the answer with an "insufficient evidence" message
 - Surface the confidence result in the UI as a visible tag
 - Re-test, record scores — including a new metric: % of answers that are fully grounded
-

Stage 6 — Getting the Foundations Right

What it does (plain language): Nothing changes visibly for the user here — this stage is about making sure what you've built is actually solid underneath, not just held together with quick fixes. You formalize how documents and their data are stored, so the system is more reliable and easier to build on going forward.

What you're building (technical):

- Move to a proper local vector database if you haven't already (Chroma)
 - Build a clear, separate "upload and process a document" step, distinct from "ask a question"
 - Write down your system's overall data flow — this becomes your architecture diagram later
-

Stage 7 — Handling Bigger, Messier Files

What it does (plain language): Real documents aren't always clean — some are huge, some are scanned images instead of typed text, some have weird formatting. This stage is about testing your system against those messier real-world files and being honest about what it can and can't handle yet, instead of it just quietly breaking.

What you're building (technical):

- Test with large and poorly formatted PDFs, note what breaks

- Add handling/limits for files that fail to parse or exceed a size limit
 - Document your real constraints (e.g. "files over X pages," "scanned/image-only PDFs not yet supported") in `decisions.md`
-

Stage 8 — Not Making People Wait

What it does (plain language): Right now, if someone uploads a big document, the whole app probably freezes until it's done processing. This stage fixes that — uploading a new document happens quietly in the background, so the app stays responsive, and you can see whether a document is still processing or ready to use.

What you're building (technical):

- Move document processing (parsing + chunking + embedding) into a background job/queue instead of doing it inline
 - Learn the basic job-queue pattern (e.g. a Redis-style worker queue)
 - Explicitly handle the case where a background job fails partway through: add a status flag (`processing` → `complete` / `failed`) and a cleanup step so half-finished, orphaned records don't quietly linger in your system
-

Stage 9 — Answers That Feel Alive

What it does (plain language): Instead of waiting for the full answer to appear all at once, the answer now types itself out word by word as it's being generated — the same feeling you get using modern AI chat tools. It makes the app feel faster and more responsive, even though the underlying speed hasn't really changed.

What you're building (technical):

- Switch your LLM calls to streaming mode
 - Update the UI to display tokens as they arrive instead of waiting for the full response
-

Stage 9.5 — Remembering the Conversation (Optional but Recommended)

What it does (plain language): Right now, every question is treated as if it's the first one ever asked — the system has no memory of what you asked before. This stage adds short-term

memory, so you can ask a follow-up question like "what about its methodology?" and the system understands you're still talking about the same paper from two questions ago, instead of getting confused.

What you're building (technical):

- Keep a short history of recent question/answer pairs
 - Feed that history into your Stage 4 query-rewriting step, so follow-up questions get rewritten with the missing context filled in before searching
 - If you choose to skip this stage, explicitly note "single-turn by design" in your Stage 11 write-up instead of leaving it silently unaddressed
-

Stage 10 — Putting It Online

What it does (plain language): Up to now, the app only works on your own computer. This stage puts it on the internet with a real link, so anyone — a friend, a recruiter, an interviewer — can open it and try it themselves without needing to install anything.

What you're building (technical):

- Deploy using a simple hosting option (Streamlit Community Cloud, Render, or Railway)
 - Store API keys securely (environment variables, not hard-coded in your code)
 - Test the live link yourself from a different device/network to confirm it actually works for a stranger
-

Stage 11 — Polish and Tell the Story

What it does (plain language): The product itself is basically done by now — this stage is about presenting it well. You put together the evidence of everything you built: proof that each feature actually helped, a short explanation of what surprised you along the way, and an honest list of what the system still can't do.

What you're building (technical):

- A results table showing Recall@K / MRR / grounding-rate before and after each stage (pulled straight from your `decisions.md` log)
- A short "what I learned / what surprised me" write-up
- A demo video showing the UI in action, including one example where it correctly says "not enough information found"

- A "known limitations / what I'd build next" section covering anything you consciously deferred (large-scale concurrency, OCR support, multi-user accounts, etc.)
-

The One Rule Across All Stages

Never start the next stage until the current one is actually working and (where relevant) tested against your Stage 1 report card. It's fine to stop after any stage — each one leaves you with something complete and demoable, not a half-finished pile of features.